

FaceLogBook

AI-assisted, contactless attendance for classes, sections, and organizations.

One-line design: React captures an image, Flask validates the attendance window, InsightFace produces a face embedding, FAISS/NumPy finds the nearest enrolled pose, and SQL records the attendance plus an audit trail.

Project Snapshot

FRONTEND React 18 + TypeScript, Vite, React Router, React Query, Tailwind/shadcn UI.	BACKEND Flask blueprints, SQLAlchemy/Alembic, JWT, rate limiting, CORS, Gunicorn-ready.
AI PATH InsightFace detection/embeddings, three pose vectors per student, FAISS with NumPy fallback.	PERSISTENCE MySQL-compatible relational database, local uploads, optional Google Drive integration.

Revision guide: Use the blue labels for implemented concepts, amber for assumptions or recommended evolution, and red for current hardening risks.
Source basis: repository code and README reviewed August 2026. Claims such as 97% accuracy and sub-100ms latency are documented targets, not independently verified here.

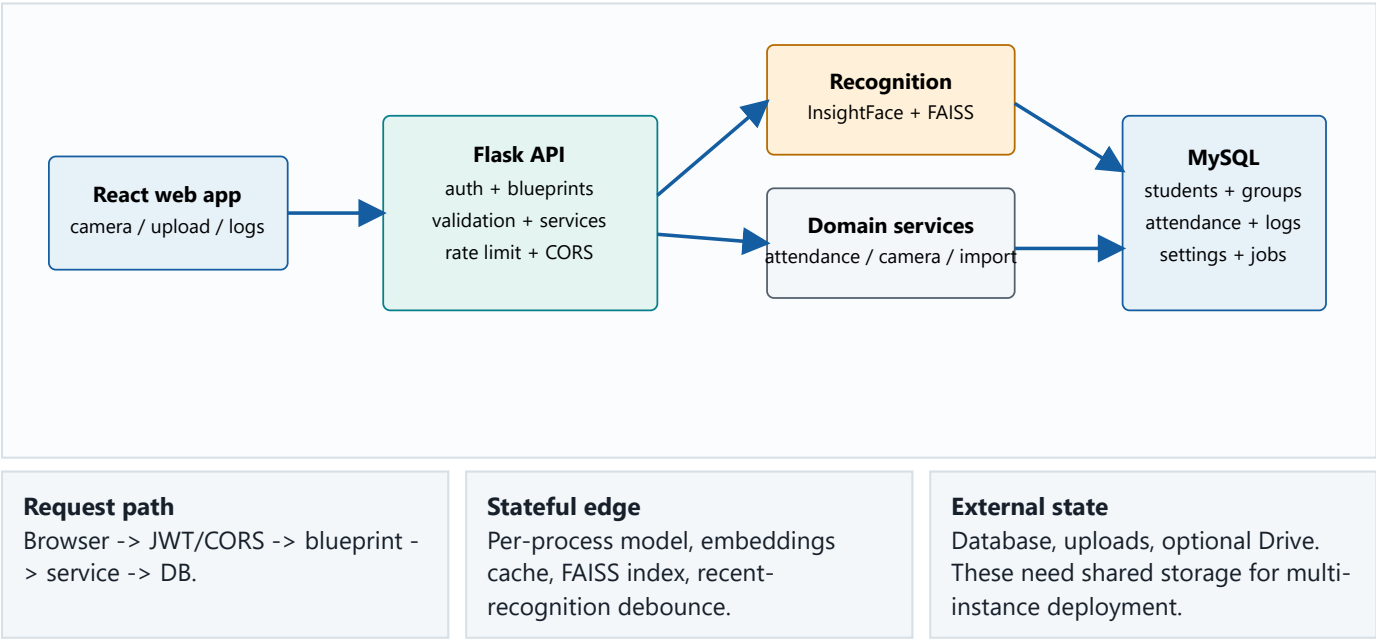
1. Features & Requirements

Key features • Live webcam and uploaded-photo recognition • On-time/late windows by group, with debounce • Student registration, multi-pose capture, CSV import • Groups/sections, bulk assignment, expiring public links • Attendance logs, camera in/out events, analytics • Recognition metrics, unrecognized-face review, settings	Functional requirements • Authenticate operators and protect admin workflows. • Recognize one or more faces and return confidence/outcome. • Reject faces outside the selected group. • Mark check-in/out idempotently within configured windows. • Persist evidence for reporting and evaluation. • Allow controlled self-registration through expiring links.
---	--

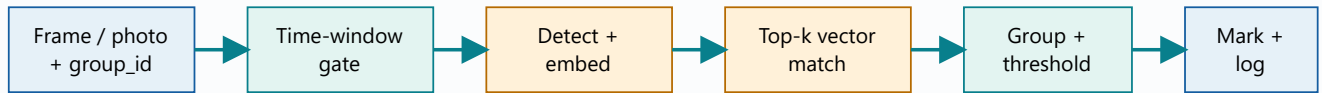
Non-functional requirements

Quality	Design response	Interview framing
Performance	Resize input, 320x320 detector, cached embeddings, FAISS/NumPy search, timing logs.	Recognition is CPU-bound; measure p95 end-to-end latency, not only search time.
Reliability	DB pool pre-ping/recycle, transactions, graceful model-init failure, health endpoints.	Model availability and database availability are separate failure domains.
Scalability	Stateless HTTP API shape; vector index cached in process; async import jobs.	Horizontal scale needs shared index refresh/versioning and external job workers.
Security	JWT, CORS allowlist, rate limiting, upload validation, expiring registration tokens.	Biometric data and authentication require stronger production controls than current dev defaults.

2. High-Level Architecture



3. Recognition Data Flow



Implemented behavior: live/upload endpoints gate by IST window, require a selected group, process recognized faces, reject wrong-section matches, then update attendance. RecognitionLog stores score, second-best score, confidence ratio, timing breakdown, threshold, source, and result.

4. Database / Schema Design

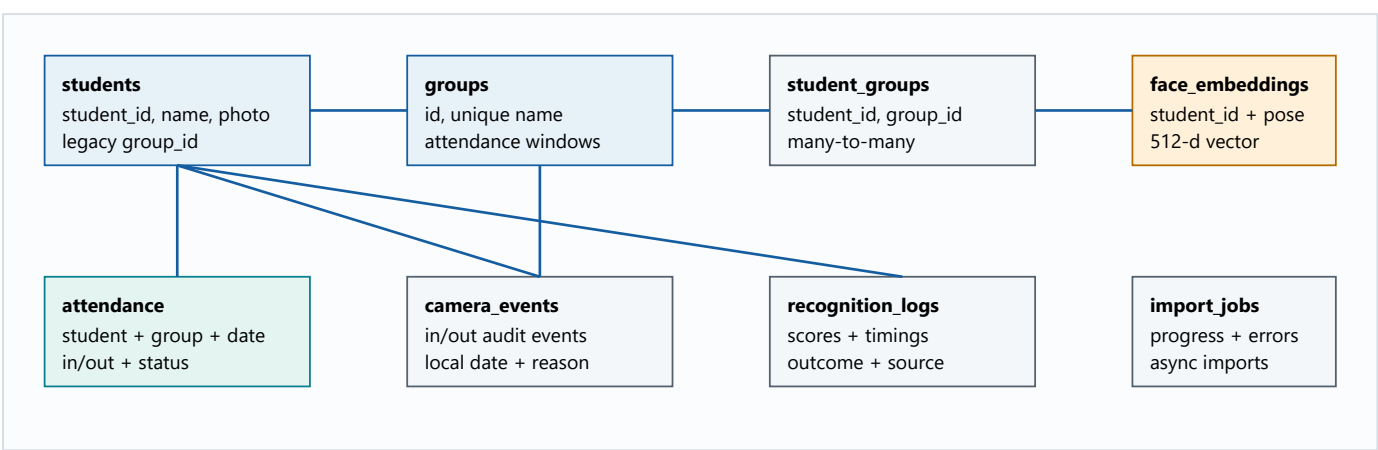


Table	Role / key constraints
students / groups	Core identity and enrollment. A legacy one-to-many group_id coexists with student_groups many-to-many; migrations/callers must handle both.
face_embeddings	Unique per student + pose (front/left/right), normalized serialized vector. Best match is mapped back to student.
attendance / camera_events	Daily business summary versus event-level in/out audit trail. Indexed by student/date and local date/timestamp.
recognition_logs	Observability and evaluation: score distribution, TP/FP/FN, latency, threshold, source.
registration_links / import_jobs / settings	Controlled onboarding, long-running import progress, and runtime attendance policy.

5. API & Service Design

Surface	Representative endpoints	Owner
Auth	/api/v1/auth/login, /refresh	JWT identity and role claims
Recognition	/api/v1/attendance/live, /upload, logs, window-status	Attendance API + FaceService + AttendanceService
Operations	/api/v1/students, /groups, /settings	CRUD, enrollment, CSV/import jobs, policies
Audit / insight	/camera-events, /metrics, /health	Event history, model timing, health/debug checks
Public onboarding	/public/register/<token>	Token validation, image checks, student registration

Service boundaries

- FaceService**: decode, resize, detect, embed, index/search, cache.
- AttendanceService**: IST windows, late policy, debounce, daily reset.
- CameraEventService**: alternating in/out events and audit metadata.
- ImportJobProcessor**: batch CSV work outside the request path.

Consistency rule

Recognition is probabilistic, but marking attendance must be deterministic: validate group, threshold, window, and debounce before the transaction commits.

Assumption: clients treat API responses as authoritative and retries may occur.

6. Scale, Reliability, Security, Performance

Scale & performance	Reliability
---------------------	-------------

- Cache embeddings/index for fast repeated reads.
- Use one vector per pose for pose robustness.
- Batch imports and move them to a durable queue at scale.
- Partition/archive recognition_logs as volume grows.
- For multiple API replicas, externalize index snapshots/cache invalidation.

Security baseline

- HTTPS, short-lived JWTs, role checks, strict CORS.
- Hash passwords with Argon2/bcrypt; rotate secrets.
- Encrypt biometric vectors and images at rest; restrict access.
- Remove debug token endpoints and avoid returning raw exceptions.
- Validate MIME, size, content, and image count; rate-limit public registration.

- Keep attendance write and event/log writes transactional where possible.
- Use health checks for DB, model, memory, and cache state.
- Store uploads in object storage with backups and lifecycle rules.
- Make import jobs resumable and idempotent.
- Use UTC storage plus explicit IST presentation consistently.

Current risks to mention honestly

HARDENING Source currently compares passwords directly, has development secret fallbacks, and exposes a debug token-check route.

ASSUMPTION README benchmark numbers need a controlled dataset, hardware profile, and p50/p95 measurement before being used as an SLO.

7. Design Decisions & Trade-offs

Decision	Why	Trade-off
Three pose embeddings	Better robustness to head angle than averaging vectors.	~3x vector storage and index entries.
FAISS with NumPy fallback	Fast local similarity search while preserving developer portability.	Fallback may be slower; index state is process-local.
Attendance + camera events	Summary queries are simple; raw events preserve auditability.	Two representations require reconciliation rules.
Group time windows	Matches classroom policy and supports late marking.	Time zone and configuration edge cases need tests.
Async import jobs	Keeps large CSV work from blocking HTTP requests.	In-process thread is not durable across restarts; use a queue for production.
Relational DB for vectors	Simple enrollment lifecycle and transactional metadata.	Dedicated vector DB/object store may fit much larger biometric workloads.

8. Interview Questions & Short Answers

Why is the system not just CRUD?

Recognition is a probabilistic, CPU-heavy pipeline. The design separates detection/matching from deterministic policy enforcement and persistence.

How do you prevent duplicate marks?

AttendanceService applies a configurable debounce and updates the existing daily record; camera events are also guarded by an event debounce.

How do you handle a wrong class?

The selected group is validated after identity matching. A recognized student outside that group is returned as wrong-section and is not marked.

What is the scaling bottleneck?

Model inference and vector index memory, followed by log volume. Scale inference separately, share/version the index, and archive or partition logs.

What happens if the model is unavailable?

Startup logs the failure and the recognition endpoint returns a service-unavailable response; health checks should surface this for orchestration.

Why keep recognition logs?

They provide latency breakdowns, threshold tuning data, confidence margins, and TP/FP/FN evaluation evidence.

What would you fix before production?

Hash credentials, remove secret fallbacks/debug disclosure, secure biometric storage, externalize jobs/cache/uploads, and add p95/load/security tests.

What assumption matters most?

The deployment is initially modest and single-process enough for an in-memory index. A multi-replica design needs explicit index synchronization and shared storage.